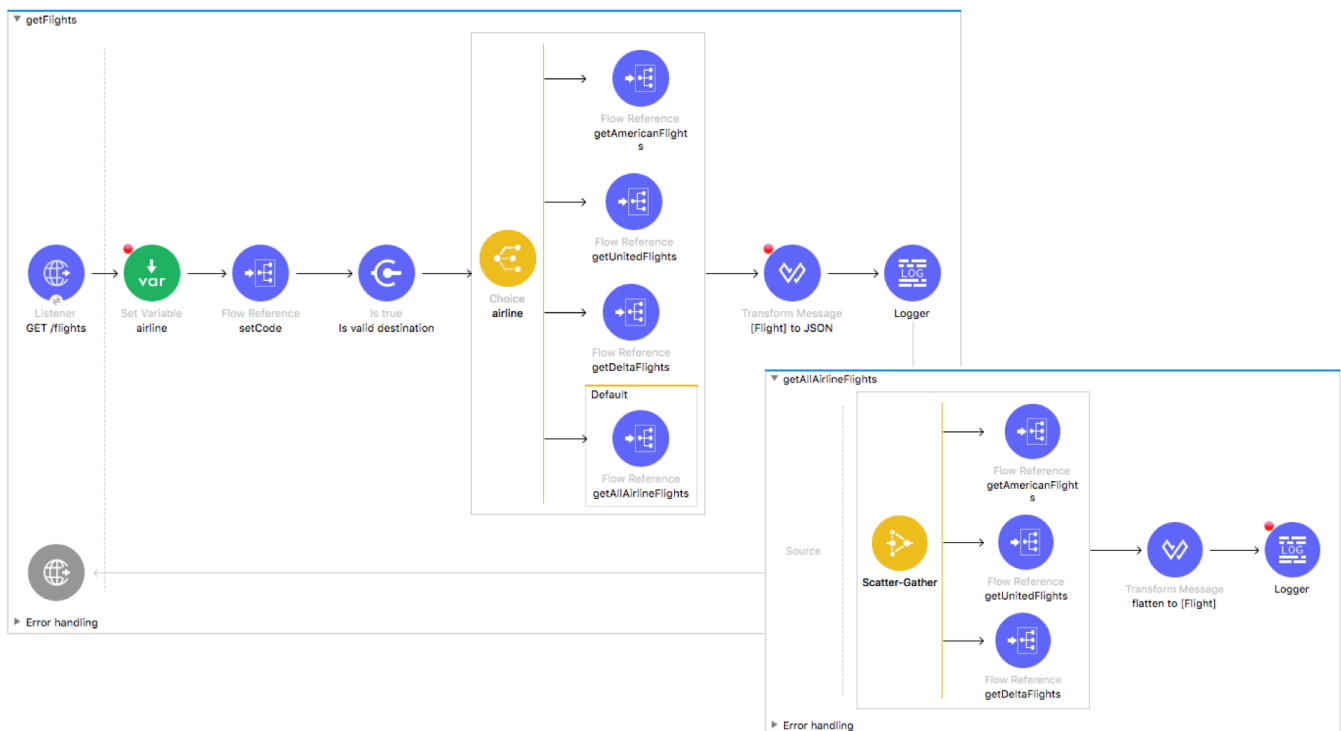


Module 9: Controlling Event Flow



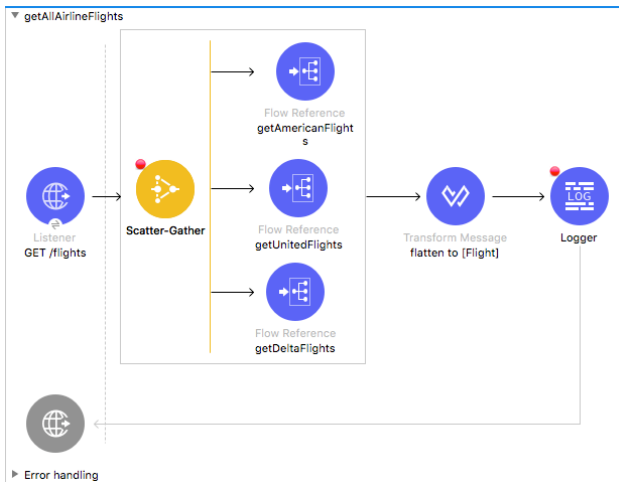
At the end of this module, you should be able to:

- Multicast events.
- Route events based on conditions.
- Validate events.

Walkthrough 9-1: Multicast an event

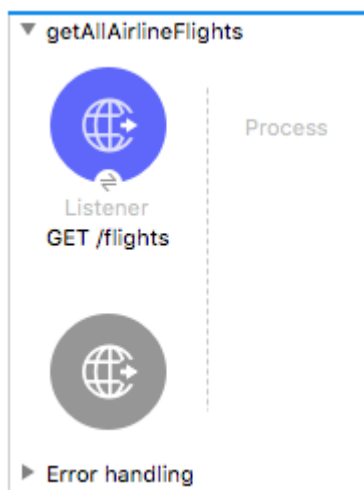
In this walkthrough, you create a flow that calls each of the three airline services and combines the results. You will:

- Use a Scatter-Gather router to concurrently call all three flight services.
- Use DataWeave to flatten multiple collections into one collection.



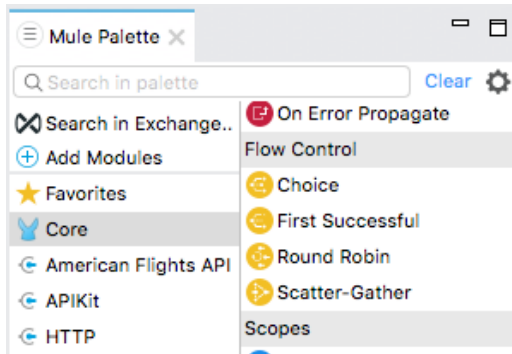
Create a new flow

1. Return to implementation.xml.
2. From the Mule Palette, drag an HTTP Listener and drop it at the top of the canvas.
3. Change the flow name to getAllAirlineFlights.
4. In the Listener properties view, set the display name to GET /flights.
5. Set the connector configuration to the existing HTTP_Listener_config.
6. Set the path to /flights and the allowed methods to GET.



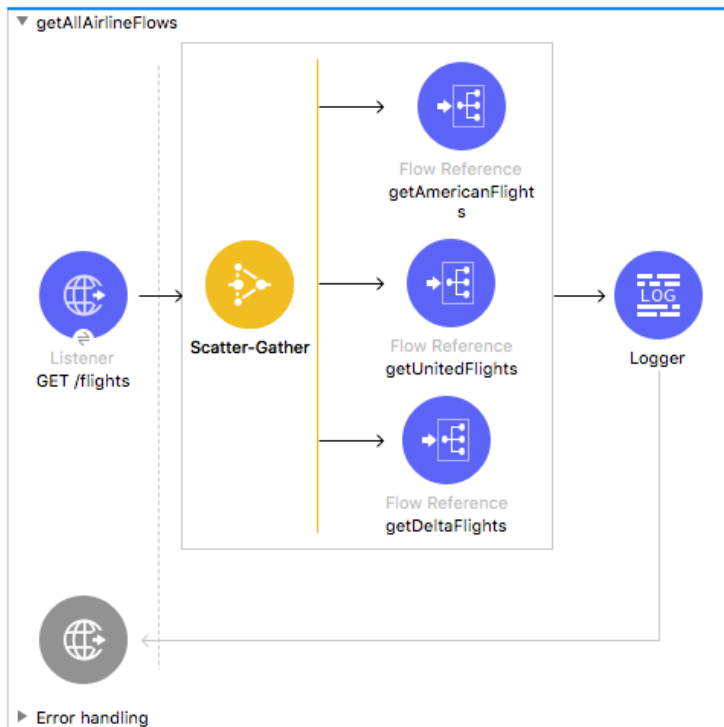
Browse the flow control elements in the Mule Palette

7. In the Core section of the Mule Palette, locate the Flow Control elements.



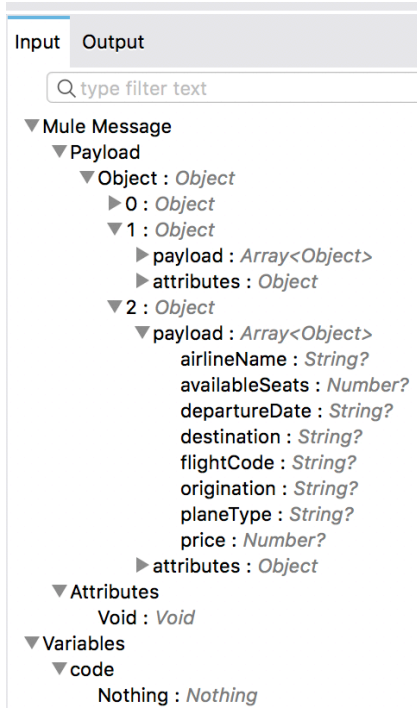
Add a Scatter-Gather to call all three airline services

8. Drag a Scatter-Gather flow control element from the Mule Palette and drop it in the process section of getAllAirlineFlights.
9. Add three Flow Reference components to the Scatter-Gather router.
10. In the first Flow Reference properties view, set the flow name and display name to getAmericanFlights.
11. Set the flow name and display name of the second Flow Reference to getUnitedFlights.
12. Set the flow name and display name of the third Flow Reference to getDeltaFlights.
13. Add a Logger after the Scatter-Gather.



Review the metadata for the Scatter-Gather output

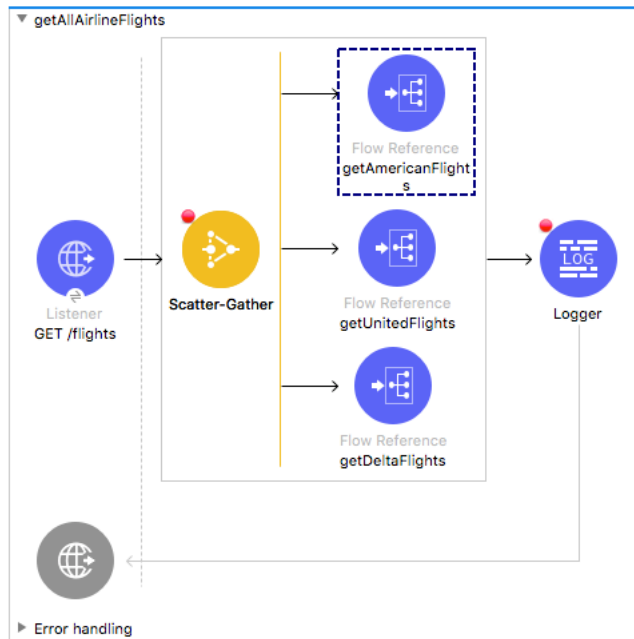
14. In the Logger properties view, explore the input payload structure in the DataSense Explorer.



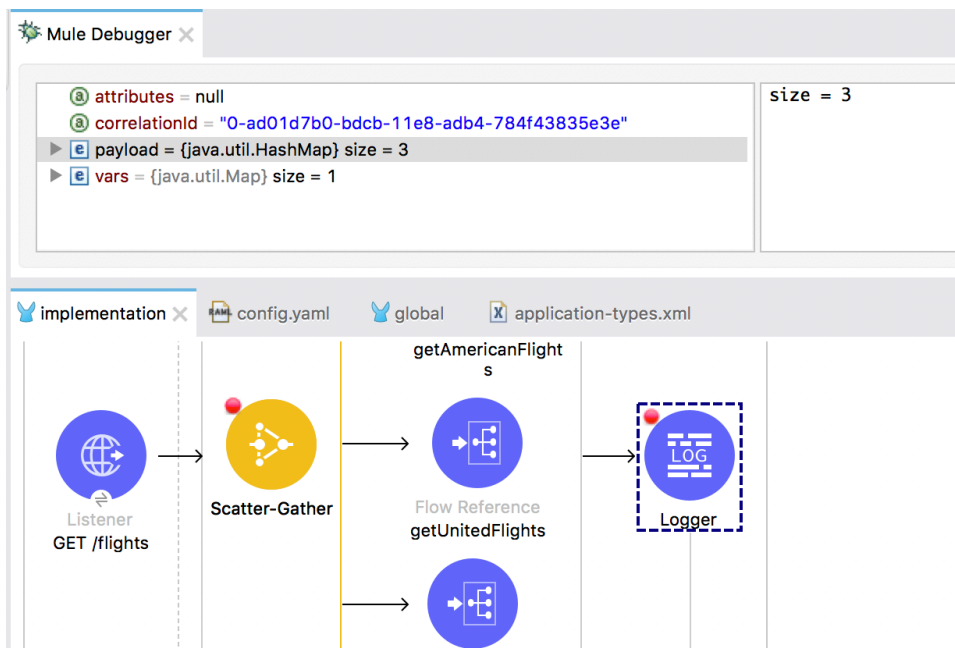
Debug the application

15. Add a breakpoint to the Scatter-Gather.
16. Add a breakpoint to the Logger.
17. Save the file to redeploy the project in debug mode.
18. In Advanced REST Client, change the URL to make a request to <http://localhost/flights>.

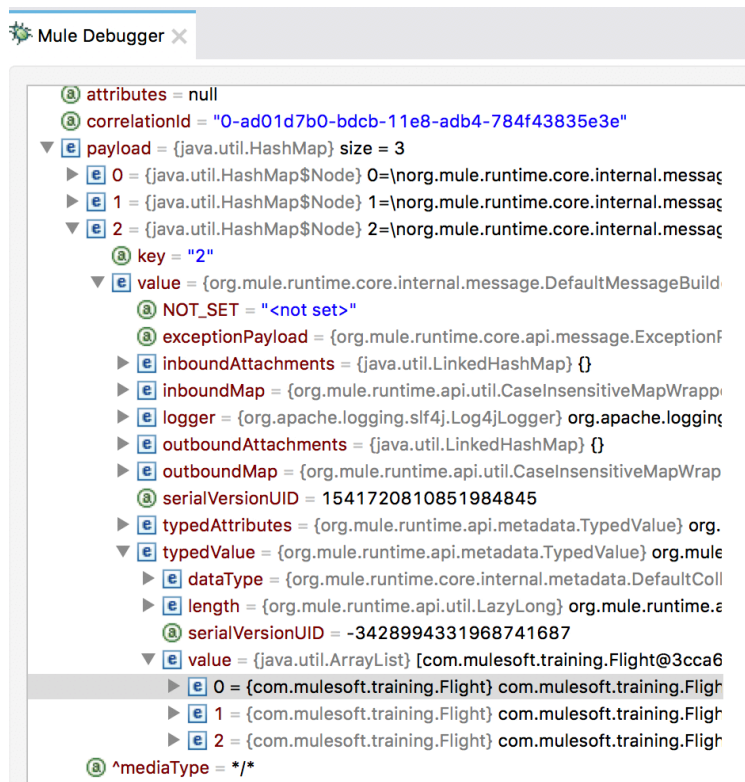
19. In the Mule Debugger, step through the application; you should step through each of the airline flows.



20. Stop at the Logger after the Scatter-Gather and explore the payload.



21. Drill-down into one of the objects in the payload.



22. Step through the rest of the application and switch perspectives.

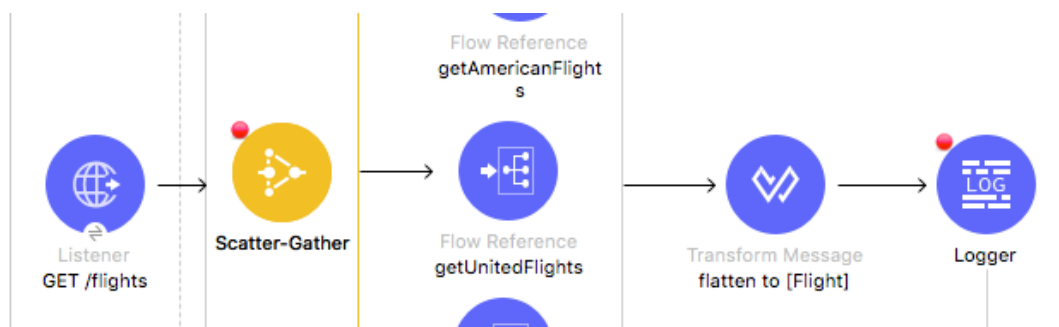
23. Return to Advanced REST Client and review the response; although the result is Java and you see a lot of characters, you should also see the Java for flight data for the three airlines.

Flatten the combined results

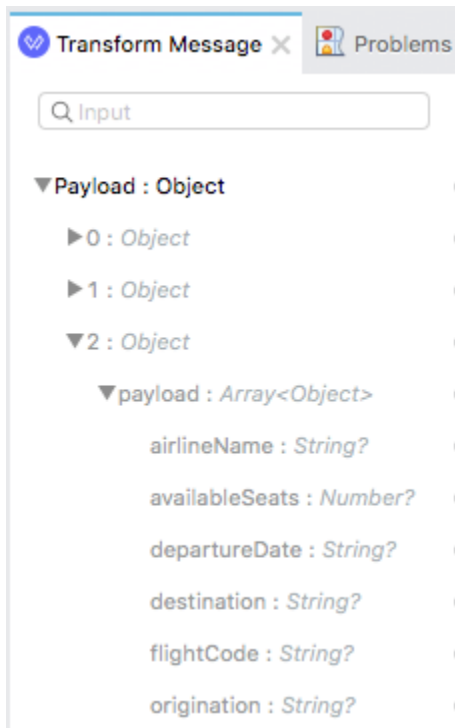
24. Return to Anypoint Studio.

25. In getAllAirlineFlights, add a Transform Message component before the Logger.

26. Change the display name to flatten to [Flight].



27. In the input section of the Transform Message properties view, review the payload data structure.



28. In the expression section, use the DataWeave flatten function to flatten the collection of objects into a single collection.

```
1 %dw 2.0
2 output application/java
3 ---
4 flatten(payload..payload)
```

Debug the application

29. Save the file to redeploy the project.
30. In Advanced REST Client, make the same request to <http://localhost:8081/flights>.

31. In the Mule Debugger, press Resume until you are stopped at the Logger at the end of the Scatter-Gather; you should see the payload is now one ArrayList of Flights.

The screenshot shows the Mule Debugger interface. The top pane displays the current state of the message:

- attributes** = null
- correlationId** = "0-81d0c140-bdcc-11e8-bd38-784f43835e3e"
- payload** = {java.util.ArrayList} size = 11
 - 0 = {com.mulesoft.training.Flight} com.mulesoft.training.Flight@2a9a1de
 - 1 = {com.mulesoft.training.Flight} com.mulesoft.training.Flight@4539aff8
 - 10 = {com.mulesoft.training.Flight} com.mulesoft.training.Flight@6f54ca2
 - 2 = {com.mulesoft.training.Flight} com.mulesoft.training.Flight@4d680dd
 - 3 = {com.mulesoft.training.Flight} com.mulesoft.training.Flight@24b3729
 - 4 = {com.mulesoft.training.Flight} com.mulesoft.training.Flight@5ebc06f
 - 5 = {com.mulesoft.training.Flight} com.mulesoft.training.Flight@3d305c8
 - 6 = {com.mulesoft.training.Flight} com.mulesoft.training.Flight@2814ed9
 - 7 = {com.mulesoft.training.Flight} com.mulesoft.training.Flight@8b7b659
 - 8 = {com.mulesoft.training.Flight} com.mulesoft.training.Flight@567a4d0
 - 9 = {com.mulesoft.training.Flight} com.mulesoft.training.Flight@49779a7
- mediaType** = application/java; charset=UTF-8

The right pane shows the **size = 11** property.

The bottom pane shows the application flow diagram:

- Listener GET /flights** (blue circle with a globe icon)
- Scatter-Gather** (yellow circle with a scatter icon)
- Flow Reference getAmericanFlights** (blue circle with a flow icon)
- Flow Reference getUnitedFlights** (blue circle with a flow icon)
- Flow Reference getDeltaFlights** (blue circle with a flow icon)
- Transform Message flatten to [Flight]** (blue circle with a transform icon)
- Logger** (blue circle with a log icon, highlighted with a dashed red box)

32. Step through the rest of the application and switch perspectives.